

Parte 2

1. Listado de tablas derivadas al Modelo Relacional

Siguiendo las reglas de derivación, cada entidad fuerte pasó a ser una tabla y las relaciones 1:N se resolvieron pasando la PK del lado "1" como clave foránea (FK) al lado "N". La relación N:M se resolvió con la tabla intermedia **pedido_producto**.

- **categoria** (**id_categoria**, nombre, activa)
- **producto** (**id_producto**, nombre, precio_lista, stock, activo, id_categoria -> categoria)
- **cliente** (**id_cliente**, nombre, email, telefono)
- **pedido** (**id_pedido**, fecha, forma_pago, id_cliente -> cliente)
- **pedido_producto** (**id_pedido** -> **pedido**, **id_producto** -> **producto**, cantidad, precio_unitario)

Justificación de la clave primaria en la tabla intermedia:

Elegí usar una clave primaria compuesta por las dos claves foráneas (**id_pedido**, **id_producto**) en lugar de crear una clave sustituta (como un **id_detalle**). Lo justifico porque, como aclara el enunciado para la Parte 3, dentro de un mismo pedido un producto no se repite en más de una línea. Por lo tanto, la combinación del número de pedido y el número de producto ya es única e identifica perfectamente cada fila. Si usara un ID inventado nuevo, de todas formas, tendría que agregarle una restricción **UNIQUE** a estas dos columnas para evitar que me carguen dos veces el mismo producto en el mismo pedido.

2. Respuestas a las preguntas guía

¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?

Si permitiera valores nulos en **id_pedido** o **id_producto**, tendría registros "huérfanos". Podría llegar a tener una fila que diga "se vendieron 2 unidades a \$1000", pero sin tener idea a qué pedido pertenecen o qué producto se le entregó al cliente. Esto rompe totalmente la integridad referencial del modelo. Además, como decidí que ambas columnas forman mi Clave Primaria compuesta, por regla fundamental de bases de datos relacionales, ningún componente de una clave primaria puede ser **NULO**.

Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto.

Sí, cambiaría la participación de la entidad **Pedido** en esa relación, pasando de ser total (obligatoria) a parcial (opcional). A nivel de la estructura del modelo relacional, esto no cambia nada (las tablas y columnas siguen siendo exactamente las mismas). El impacto real se da en las restricciones lógicas y en el manejo de la información, ahora el sistema tiene que aceptar que existan registros en la tabla **pedido** que no existan en **pedido_producto**. Esto obligaría a implementar controles extra (por ejemplo, a nivel de la aplicación o con un campo de "estado" del pedido) para evitar que un pedido vacío pase a estado facturado o se intente cobrar.

Parte 3

1. Clave Primaria (PK)

Analizando la planilla, me doy cuenta de que ni el **N.º pedido** por sí solo (se repite cuando un pedido tiene varios productos) ni el **Producto** por sí solo (se repite en varios pedidos distintos) sirven para identificar una fila única. Como la regla dice que un producto no se repite en más de una línea dentro de un mismo pedido, la única forma de identificar cada registro de manera unívoca es combinando ambos.

- Clave Candidata / Primaria elegida: {**N.º pedido, Producto**}.

2. Dependencias Funcionales (FD)

Basándome en los datos y las reglas de negocio del enunciado, identifiqué las siguientes dependencias funcionales:

- DF1: **N.º pedido** -> **Fecha, Cliente, Forma de pago** (Un pedido tiene una única fecha, cliente y forma de pago).
- DF2: **Producto** -> **Categoría** (Cada producto pertenece a una sola categoría).
- DF3: {**N.º pedido, Producto**} -> **Precio unitario, Cantidad, Subtotal** (El precio histórico congelado, la cantidad y el subtotal dependen exactamente de qué producto se vendió en qué pedido).
- DF4: {**Cantidad, Precio unitario**} -> **Subtotal** (El subtotal es un campo calculable).

3. Verificación de 1FN

Para que la tabla esté en Primera Forma Normal (1FN), todos los atributos deben ser atómicos (no tener listas ni grupos repetidos en una misma celda).

- **Análisis:** La planilla plana que nos pasaron ya cumple con esto. Cada celda tiene un solo valor.
- **Resultado:** Cumple 1FN. La relación universal sigue siendo la misma, con la PK compuesta {**N.º pedido, Producto**}.

4. Verificación de 2FN

La Segunda Forma Normal (2FN) exige estar en 1FN y que ningún atributo no clave dependa solo de una parte de la clave compuesta (es decir, no debe haber dependencias parciales).

- **Análisis:** Acá hay violaciones a la 2FN. Según mi DF1, la Fecha, el Cliente y la Forma de pago dependen únicamente del **N.º pedido** (una parte de la clave). Según mi DF2, la Categoría depende únicamente del **Producto** (la otra parte de la clave).
- **Corrección:** Separo estas dependencias parciales en sus propias tablas, dejando en la tabla original solo lo que depende de la clave compuesta completa.

- Tablas en 2FN:
 - Pedido (**N.º pedido, Fecha, Cliente, Forma de pago**)
 - Producto (**Producto, Categoría**)
 - Detalle_Pedido (**N.º pedido, Producto, Precio unitario, Cantidad, Subtotal**)

5. Verificación de 3FN

La Tercera Forma Normal (3FN) exige estar en 2FN y que no existan dependencias transitivas (ningún atributo no clave debe depender de otro atributo no clave).

- **Análisis:** En la tabla **Detalle_Pedido**, el atributo **Subtotal** depende matemáticamente de **Cantidad y Precio unitario** (mi DF4). Como estoy guardando un dato que puedo calcular a partir de otros dos atributos no clave, hay una dependencia transitiva.
- **Corrección:** Elimino la columna **Subtotal** para evitar la redundancia y cumplir la 3FN. Las tablas **Pedido** y **Producto** ya cumplen porque no tienen dependencias transitivas entre sus campos.
- Tablas en 3FN:
 - Pedido (**N.º pedido, Fecha, Cliente, Forma de pago**)
 - Producto (**Producto, Categoría**)
 - Detalle_Pedido (**N.º pedido, Producto, Precio unitario, Cantidad**)

6. Verificación de BCNF

La Forma Normal de Boyce-Codd (BCNF) establece que para toda dependencia funcional no trivial $X \rightarrow Y$, el determinante X debe ser una clave candidata (superclave).

- **Análisis:**
 - En **Pedido**, el determinante de todos los atributos es **N.º pedido**, que es la clave primaria. Cumple.
 - En **Producto**, el determinante es **Producto**, que es la clave primaria. Cumple.
 - En **Detalle_Pedido**, el determinante de cantidad y precio es **{N.º pedido, Producto}**, que es la clave primaria. Cumple.
- **Resultado:** No encontré ninguna excepción. Todas las tablas ya se encuentran en BCNF porque todos los determinantes son claves primarias.

7. Conjunto final de tablas normalizadas

- pedido (**nro_pedido, fecha, cliente, forma_pago**)
- producto (**producto, categoria**)
- pedido_producto (**nro_pedido, producto, precio_unitario, cantidad**)

Preguntas de Integración

Comparé las tablas que obtuviste normalizando la planilla con las tablas que derivaste en la Parte 2 a partir de tu modelo ER...

Las tablas obtenidas por normalización coinciden estructuralmente con las entidades de mi diagrama ER: tengo una tabla para los **Pedidos**, otra para los **Productos** y una intermedia para la relación de muchos a muchos. Sin embargo, hay diferencias importantes. En mi ER original modelé entidades fuertes e independientes para **Cliente** y **Categoría** (con sus propios IDs), mientras que, al normalizar esta planilla plana, "**Categoría**" quedó rebajada a un simple atributo de Producto, y "**Cliente**" a un atributo de Pedido. Además, en la normalización perdí campos valiosos que no estaban en la planilla original (como el email, teléfono, stock y las marcas de baja). Esto me demuestra que normalizar a partir de datos históricos planos sirve para estructurar el pasado, pero modelar desde cero a partir de un requerimiento de negocio (el enunciado ER) te permite crear una base de datos mucho más rica, escalable y representativa de la realidad.

El atributo subtotal de cada línea es, en teoría, calculable como $\text{cantidad} * \text{precio_unitario}$. ¿Conviene almacenarlo igual, o siempre recalcularlo al vuelo? ¿Esa decisión de diseño viola alguna forma normal...?

Conviene no almacenarlo y calcularlo siempre en el momento. Almacenarlo viola la Tercera Forma Normal (**3FN**) porque crea una dependencia transitiva (un atributo no clave dependiendo de otros dos atributos no clave). Guardarlo genera redundancia de datos y un riesgo grave de inconsistencia: si por algún error en el sistema se actualiza la cantidad de un producto, pero no se actualiza el subtotal guardado, la base de datos quedaría corrupta. Como es un simple cálculo matemático ($\text{cantidad} * \text{precio}$), el motor lo resuelve instantáneamente en las consultas, por lo que no se justifica comprometer la normalización.